

Correction des Questions Théoriques – TP 2

Guidage par Vision Artificielle et Intégration Système sous ROS

2

Filière Robotique

Mai 2026

Table des matières

1	Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)	2
1.1	Question 1 : Problème du drapeau <code>cv2.THRESH_BINARY</code> et correction	2
1.2	Question 2 : Formules mathématiques des moments et signification du rapport . .	2
1.3	Question 3 : Instabilité et déplacement saccadé sous contrôle Proportionnel (P) .	2
2	Correction de la Partie 2 : Éléments Théoriques	3
2.1	Question 1 : Analyse architecturale et portée des variables (<i>Scope</i>)	3
2.2	Question 2 : Importance de la gestion du temps dynamique (<i>dt</i>)	3
2.3	Question 3 : Analyse comportementale et rôle du terme Dérivé	3

1 Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)

1.1 Question 1 : Problème du drapeau `cv2.THRESH_BINARY` et correction

La fonction `cv2.threshold` configurée avec le drapeau `cv2.THRESH_BINARY` transforme tous les pixels dont l'intensité est supérieure au seuil (200) en blanc (valeur maximale 255), et tous les autres en noir (0).

Analyse de l'erreur : La piste à suivre est constituée de **lignes noires sur un sol clair**. Le sol clair possède une intensité lumineuse élevée (> 200) et sera donc converti en blanc (255), tandis que la ligne noire (< 200) sera convertie en noir (0). L'algorithme `cv2.moments` calcule ses intégrales spatiales en se basant sur l'intensité des pixels. Ici, il va calculer le centre de gravité de la zone blanche, c'est-à-dire du **sol clair**, faussant totalement la détection de la trajectoire.

Modification à apporter : Il faut inverser la logique binaire en utilisant le drapeau `cv2.THRESH_BINARY_INV`. Ainsi, la ligne noire prendra la valeur de surbrillance 255 (blanc) et le sol deviendra noir (0), permettant aux moments géométriques de se focaliser exclusivement sur la ligne de guidage.

1.2 Question 2 : Formules mathématiques des moments et signification du rapport

Pour une image discrète binarisée $I(x, y)$ au sein de la Région d'Intérêt (ROI), les formules des moments géométriques d'ordre 0 et 1 sont définies par :

$$M_{00} = \sum_x \sum_y I(x, y) \quad (1)$$

$$M_{10} = \sum_x \sum_y x \cdot I(x, y) \quad (2)$$

Signification physique :

- M_{00} représente la somme des intensités de tous les pixels blancs de l'image segmentée, ce qui équivaut physiquement à **l'aire de la ligne** (sa masse de pixels) dans la ROI.
- Le rapport $\frac{M_{10}}{M_{00}}$ définit mathématiquement la **coordonnée horizontale C_x du centre de gravité (barycentre)** de la ligne noire segmentée dans le repère de l'image :

$$C_x = \frac{M_{10}}{M_{00}} \quad (3)$$

1.3 Question 3 : Instabilité et déplacement saccadé sous contrôle Proportionnel (P)

Un contrôleur purement proportionnel génère une vitesse de rotation ω_z directement proportionnelle à l'erreur de centrage ($\omega_z = k_p \cdot e$). Ce système souffre d'un **défaut d'amortissement intrinsèque** (facteur d'amortissement nul ou très faible) :

- Lorsque le robot s'écarte de la ligne, il applique un fort couple pour y revenir.
- À cause de son inertie mécanique, lorsqu'il croise l'axe de la ligne, son erreur est nulle mais sa vitesse angulaire est maximale. Le robot **dépasse alors la ligne** et bascule de l'autre côté.
- Ce comportement crée des oscillations permanentes en lacet de gauche à droite. Ces variations violentes d'attitude perturbent la caméra, modifient continuellement l'erreur de calcul et rendent le déplacement linéaire instable et saccadé.

2 Correction de la Partie 2 : Éléments Théoriques

2.1 Question 1 : Analyse architecturale et portée des variables (*Scope*)

Problématique identifiée :

Dans le script initial, les variables `gain_kp` et `gain_kd` sont déclarées directement dans le corps du constructeur `__init__()`. En Python, cela en fait des **variables locales** à cette fonction. Dès que l'initialisation du nœud ROS 2 se termine, l'espace mémoire de ces variables est libéré.

Lorsque le timer appelle périodiquement la méthode `boucle()`, le script tente d'accéder à ces variables qui n'existent plus dans son espace de nommage (*namespace*), ce qui provoque un crash immédiat du nœud avec l'erreur système suivante :

```
NameError: name 'gain_kp' is not defined
```

Solution attendue :

Pour rendre ces paramètres persistants et accessibles depuis n'importe quelle méthode de la classe (notamment la boucle de contrôle), il est impératif de les lier à l'instance de l'objet en utilisant le mot-clé `self`.

```
# Dans le __init__() :
self.kp = 0.002
self.kd = 0.001

# Dans le boucle() :
P_Correcteur = self.kp * self.error
```

2.2 Question 2 : Importance de la gestion du temps dynamique (*dt*)

Il est mathématiquement et physiquement erroné d'utiliser une valeur de temps fixe (ex : 0.1 s) dans un système robotique asynchrone pour deux raisons majeures :

1. **Le jitter de l'OS (Goulot d'étranglement)** : Ubuntu et ROS 2 s'exécutent par défaut sur un noyau non-temps-réel strict. Selon la charge du processeur (notamment si le traitement d'images de la caméra prend plus de ressources sur une frame), l'intervalle réel entre deux appels du timer fluctue continuellement (ex : 0.092 s, puis 0.118 s).
2. **La stabilité du calcul de la dérivée** : Le terme Dérivé repose sur l'approximation discrète de la pente de l'erreur :

$$D = K_d \cdot \frac{\Delta e}{\Delta t}$$

Si Δt réel varie mais que le code utilise une constante théorique, le calcul de la vitesse de variation de l'erreur devient faux. Cela introduit un bruit numérique important qui se traduit par des commandes de vitesse hachées, provoquant des saccades et une usure prématurée des moteurs.

2.3 Question 3 : Analyse comportementale et rôle du terme Dérivé

Réaction à l'entrée d'une courbe :

Lors de l'approche d'un virage brusque à 90°, l'erreur de suivi de ligne augmente de manière extrêmement brutale.

- Le terme **Proportionnel** ($P = K_p \cdot e$) ne réagit qu'à l'amplitude instantanée de l'erreur. Au début du virage, l'erreur étant encore faible, son action est insuffisante.

- Le terme **Dérivé** ($D = K_d \cdot \frac{de}{dt}$) réagit à la **vitesse de variation** de cette erreur. Comme la pente de l'erreur est abrupte, la dérivée prend immédiatement une valeur très élevée.

Effet d'anticipation et d'amortissement :

Le terme Dérivé agit comme un système d'anticipation : il injecte une correction angulaire massive *avant même* que l'erreur proportionnelle n'atteigne son maximum, permettant au robot de "plonger" efficacement dans le virage.

De plus, lorsque le robot a fini de pivoter et se réaligne vers le centre de la piste, l'erreur diminue rapidement ($\frac{de}{dt} < 0$). Le terme Dérivé change alors de signe et s'oppose à la rotation. Il agit comme un **amortisseur visqueux virtuel**, freinant le robot pour l'empêcher de dépasser sa cible (*overshoot*) et supprimant ainsi l'effet de *zig-zag* (oscillations autour de la ligne).